Smash Battle

Где и в каком файле реализована каждая часть игры

Назначение документа

Это последовательная карта проекта для защиты лабораторной работы. Открывайте файлы в указанном порядке: каждый следующий класс получает результат предыдущего и отвечает только за одну область решения.

1	Создание	Program -> composition root -> use case -> builder -> factories
2	Ход	initiative -> stats -> reachable area -> input
3	Действие	command -> service -> domain -> event -> log
4	Продолжение	AI -> victory -> save/load -> render

Главная мысль архитектуры

Domain хранит состояние. Application выполняет игровые правила. Creation создаёт объекты. Infrastructure работает с Windows, JSON и случайностью. Presentation рисует и переводит ввод в команды. Bootstrap только соединяет зависимости.

Короткая формула для преподавателя: ввод не меняет персонажа напрямую. Он создаёт команду, команда вызывает отдельный сервис, сервис изменяет Domain и публикует событие.

Корень решения

```
SmashBattle.sln
```

Исполняемый проект: src/SmashBattle.Console. Тесты: tests/SmashBattle.Tests.

1. От запуска программы до готового боя

Шаг	Файл / класс	Что происходит
1	`Program.cs`	Настраивает UTF-8, заголовок консоли и передаёт запуск в GameCompositionRoot.Create().Run(). Бизнес-логики здесь нет.
2	`Bootstrap/GameCompositionRoot.cs`	Создаёт GameState, путь сохранения, BattleRuntimeFactory и четыре use case главного меню.
3	`Presentation/Menus/MainMenu.cs`	Показывает пункты Play, состав, главы, паттерны и Exit. При выборе Play вызывает StartBattleUseCase.
4	`Application/UseCases/StartBattleUseCase.cs`	Получает настройки, просит фабрику создать runtime, сохраняет последний бой в GameState и запускает BattleLoop.
5	`Presentation/Menus/BattleSetupReader.cs`	Читает игровой клан. Seed можно ввести вручную или оставить пустым.
6	`Infrastructure/Randomness/RandomSeedGenerator.cs`	Создаёт случайный seed, когда пользователь нажал Enter на пустой строке.
7	`Bootstrap/BattleRuntimeFactory.cs`	Composition root одного боя. Соединяет карту, фабрики, сервисы, AI, renderer, input, журнал и сохранение.

Что передаётся дальше

```
playerClan + seed -> BattleRuntimeFactory.Create(...) -> BattleBuilder.Build(...) -> BattleState
```

Важно: BattleRuntimeFactory большой по количеству строк создания объектов, но он не содержит игровых условий. Его единственная ответственность - связать реализации с интерфейсами.

Где лежат базовые настройки

Шаг	Файл / класс	Что происходит
1	`Creation/Combatants/CombatantPrototypeCatalog.cs`	HP, урон, защита, скорость, крит, уклонение, инициатива, способность и аура каждой расы.
2	`Creation/Equipment/WeaponCatalog.cs`	Список оружия, дальность и бонус урона.
3	`Creation/World/TerrainCatalog.cs`	Символ, цена движения и проходимость каждого типа местности.
4	`Presentation/Animation/MovementAnimationSettings.cs`	Задержка между отдельными шагами маршрута.

2. Создание карты, кланов и персонажей

Шаг	Файл / класс	Что происходит
1	`Creation/Battle/BattleBuilder.cs`	Задаёт размер карты 50x50, создаёт три зоны появления и собирает BattleState.
2	`Creation/World/MapGenerator.cs`	Для каждой координаты выбирает plain, forest, rocks, water или hill. Зоны появления очищает до равнины.
3	`Creation/Clans/ClanFactoryProvider.cs`	По ClanType возвращает нужную фабрику, не раскрывая BattleBuilder конкретные классы.
4	`ElfisClanFactory.cs` / `DwargonClanFactory.cs` / `WrumperClanFactory.cs`	Каждая конкретная фабрика переопределяет CreatePrototype() и выбирает шаблон своей расы.
5	`Creation/Clans/ClanFactoryBase.cs`	Выбирает 3-5 бойцов, случайного главу, имя, оружие и инициативу. Затем клонирует прототип.
6	`Creation/Combatants/CombatantPrototype.cs`	Создаёт уникальный Combatant. Главе добавляет +50% HP, +20% урона, +2 инициативы и ауру.
7	`Creation/Names/GeneratedNameProvider.cs`	Комбинирует расовые префиксы и суффиксы и не повторяет использованные имена.
8	`Creation/Clans/ClanLeadershipRegistry.cs`	Singleton-реестр хранит по одному текущему главе каждого клана.

Domain после создания

Шаг	Файл / класс	Что происходит
1	`Domain/Battle/BattleState.cs`	Карта, три клана, выбранный игроком клан, seed, раунд и индекс хода.
2	`Domain/Clans/Clan.cs`	Тип клана, список участников, ссылка на единственного главу и признак, остался ли клан жив.
3	`Domain/Combatants/Combatant.cs`	HP, позиция, очки движения, cooldown, stun, состояние и безопасные методы изменения этих значений.
4	`Domain/World/BattleMap.cs`	Хранит TerrainType[,] и отвечает только на вопросы: входит ли координата в карту и какая там местность.

```
BattleBuilder -> MapGenerator + IClanFactory[] -> BattleState
```

3. Инициатива, начало хода и движение

Шаг	Файл / класс	Что происходит
1	`Application/Turns/TurnScheduler.cs`	Сортирует бойцов по инициативе, скорости, HP и имени. Хранит CurrentIndex и начинает ход следующего живого персонажа.
2	`Application/Stats/EffectiveStatsCalculator.cs`	Берёт базовые характеристики и последовательно пропускает их через список IStatsModifier.
3	`Domain/Combatants/Combatant.BeginTurn()`	Снижает cooldown, обрабатывает stun, добавляет эффективную скорость к MovementPoints и освобождает основное действие.
4	`Application/Battle/BattleSession.cs`	Соединяет текущий BattleState, TurnScheduler, BattleLog, режим выбора и рассчитанную область движения.
5	`Application/Movement/ReachableAreaFinder.cs`	Алгоритмом Дейкстры обходит соседние клетки, исключает воду и занятые позиции, учитывает доступные MovementPoints.
6	`Application/Movement/MovementCostCalculator.cs`	Возвращает цену клетки. Расовая местность стоит 2.0, обычная равнина 2.5, вода непроходима.
7	`Application/Movement/ReachableArea.cs`	Хранит цену каждой достижимой клетки и Previous для восстановления полного маршрута.

Что происходит после клика или WASD

Шаг	Файл / класс	Что происходит
1	`Application/Commands/MoveCommand.cs`	Передаёт текущего персонажа, конечную координату и ReachableArea в MovementService.
2	`Application/Movement/MovementService.cs`	Восстанавливает PathTo(destination), списывает цену каждого шага и меняет позицию по одной клетке.
3	`Presentation/Animation/ConsoleMovementStepObserver.cs`	После каждого шага обновляет область, перерисовывает карту и ждёт заданную задержку.
4	`Application/Events/MovementCompletedEvent.cs`	После последней клетки публикует итог: кто, откуда, куда и за какую стоимость прошёл.

```
Input -> MoveCommand -> MovementService -> каждый шаг -> renderer -> MovementCompletedEvent
```

Почему гном иногда не идёт в первый ход: его скорость 2, а обычная клетка стоит 2.5. Очки не пропадают, поэтому на следующем ходу накопленного значения уже достаточно.

4. Ввод мышью, клавиатура и интерфейс

Шаг	Файл / класс	Что происходит
1	`Infrastructure/ConsoleInput/WindowsConsoleInputReader.cs`	Клавиши читает через Console.ReadKey, мышь - через Win32 ReadConsoleInput. Возвращает единый GameInput.
2	`Presentation/Input/GameInput.cs`	Простой объект: тип ввода, ConsoleKey и координаты мыши.
3	`Presentation/Controllers/PlayerTurnController.cs`	Записывает Last input и маршрутизирует событие в mouse-controller или keyboard-controller.
4	`PlayerKeyboardController.cs`	WASD/стрелки, Space, Q, E, Tab, P, L, F5, F9, X и Esc. При ошибке публикует понятное сообщение в журнал.
5	`PlayerMouseController.cs`	Проверяет HUD-кнопки через HitTest или переводит экранную координату карты в мировую.
6	`Application/Commands/CommandDispatcher.cs`	Получает любой IPlayerCommand и вызывает Execute(). Мышь и клавиатура используют одинаковые команды.

Рендеринг разделён на четыре части

Шаг	Файл / класс	Что происходит
1	`Presentation/Rendering/ConsoleBattleRenderer.cs`	Координатор: очищает экран, рисует заголовок и вызывает три специализированных renderer.
2	`ConsoleMapRenderer.cs`	Карта, символы бойцов, контуры доступных клеток, зоны способностей и аур.
3	`ConsoleHudRenderer.cs`	Текущий персонаж, HP, movement, оружие, action, cooldown, target, Last input и кнопки.
4	`ConsoleBattleLogRenderer.cs`	Последние восемь событий на всю ширину под картой.
5	`BattleLogViewer.cs`	Полный журнал с прокруткой после нажатия L.

Если кажется, что клавиша не работает, сначала смотрим Last input. Если клавиша показана, ввод дошёл; причина отказа действия будет в нижнем журнале.

5. Обычная атака и расчёт урона

Шаг	Файл / класс	Что происходит
1	`Application/Commands/AttackCommand.cs`	Хранит BattleSession, CombatService и выбранную цель. Execute() передаёт их в сервис.
2	`Application/Combat/CombatService.cs`	Проверяет дальность, живы ли участники, разные ли кланы и доступно ли действие.
3	`EffectiveStatsCalculator.cs`	Повторно считает актуальные характеристики атакующего и цели с учётом ауры и местности.
4	`CombatService.Attack()`	Сначала проверяет уклонение. При успехе создаёт DamageContext и передаёт его в DamagePipeline.
5	`Application/Combat/RollDamageStage.cs`	Случайное число между min/max damage плюс бонус оружия.
6	`CriticalDamageStage.cs`	Проверяет шанс критического удара и умножает урон на 1.5.
7	`DefenseDamageStage.cs`	Вычитает защиту цели. Defending дополнительно даёт +5 защиты.
8	`MinimumDamageStage.cs`	Не позволяет итоговому урону стать меньше 1.
9	`Combatant.ReceiveDamage()`	Снимает HP и переводит состояние в Dead, когда здоровье стало нулём.
10	`AttackResolvedEvent.cs`	Формирует полную строку: атакующий, цель, урон, источник, крит и оставшееся HP.

Цепочка ответственности

```
RollDamageStage -> CriticalDamageStage -> DefenseDamageStage -> MinimumDamageStage
```

На защите удобно показать, что новая стадия - например, бронепробитие - добавляется отдельным классом между CriticalDamageStage и DefenseDamageStage, а не переписывает CombatService.

6. Способности, защита, ауры и местность

Шаг	Файл / класс	Что происходит
1	`Application/Commands/AbilityCommand.cs`	Передаёт центр выбранной области в AbilityService.
2	`Application/Combat/AbilityService.cs`	Проверяет cooldown и range, собирает врагов внутри AreaRadius и применяет попадание к каждому.
3	`CombatantPrototypeCatalog.cs`	Whirlwind: соседняя область. Arrow Rain: дальность 6 и радиус 2. Ground Slam: урон и stun.
4	`Application/Commands/DefendCommand.cs`	Передаёт текущего бойца в DefendService.
5	`Application/Combat/DefendService.cs`	Тратит действие, переводит персонажа в Defending и публикует событие.
6	`Application/Auras/AuraService.cs`	При старте и каждом цикле проверяет расстояние до живого главы и управляет временным бонусом максимального HP.
7	`Application/Stats/AuraStatsModifier.cs`	Добавляет damage, defense, speed, dodge и max HP. Не усиливает самого главу.
8	`Application/Stats/TerrainStatsModifier.cs`	Elfis в лесу получают dodge, Dwargon на камнях defense, Wrumper на равнине damage, стрелок на холме damage.

Где меняются значения

```
HP, damage, speed, crit, dodge, ability, aura -> CombatantPrototypeCatalog.cs

range и damage bonus оружия -> WeaponCatalog.cs

цена и символ местности -> TerrainCatalog.cs
```

7. AI, события, журнал и сохранение

Шаг	Файл / класс	Что происходит
1	`Application/Battle/BattleLoop.cs`	Главный цикл. Рисует экран, обрабатывает stun/паузу, передаёт ход игроку или AI и проверяет победу.
2	`Application/AI/AiControllerProvider.cs`	По ClanType выбирает Aggressive, Ranged или Defensive controller.
3	`Application/AI/AiTurnTemplate.cs`	Template Method: цель -> способность -> атака -> движение -> повторная атака -> защита.
4	`AggressiveAiController.cs`	Wrumper выбирает ближайшего врага.
5	`RangedAiController.cs`	Elfis предпочитает раненую цель и сохраняет дистанцию.
6	`DefensiveAiController.cs`	Dwargon выбирает угрозу, которая находится ближе всего к главе.
7	`Application/Events/BattleEventPublisher.cs`	Observer publisher передаёт IBattleEvent всем подписчикам.
8	`Application/Logging/BattleLog.cs`	Подписчик хранит полную историю. HUD берёт последние восемь записей.
9	`Application/Persistence/BattleMementoFactory.cs`	F5 снимает состояние бойцов, раунда, хода и генератора случайности; F9 восстанавливает.
10	`Infrastructure/Persistence/BattleSaveStore.cs`	Сериализует BattleMemento в JSON в LocalApplicationData/SmashBattle.

Почему сохраняется RandomState

Без состояния генератора после F9 следующие критические удары и уклонения отличались бы от сохранённой ветки. SeededRandomSource.Restore() возвращает точную последовательность будущих бросков.

Условие победы

Application/Battle/VictoryCondition.cs возвращает ClanType только тогда, когда жив ровно один клан.

8. Карта паттернов

Паттерн	Где смотреть	Зачем
Abstract Factory	`IClanFactory + concrete clan factories`	BattleBuilder получает семейство клана через абстракцию.
Factory Method	`ClanFactoryBase.CreatePrototype()`	Каждая раса переопределяет создание своего прототипа.
Prototype	`CombatantPrototype.Clone()`	Из шаблона создаются уникальные бойцы.
Builder	`BattleBuilder`	Пошагово собирает карту, кланы и реестр глав.
Singleton	`ClanLeadershipRegistry`	Один реестр глав на текущую игру.
Adapter	`WindowsConsoleInputReader`	Мышь Win32 и ConsoleKey становятся GameInput.
Decorator	`IStatsModifier`	Аура и местность наслаиваются на базовые stats.
Flyweight	`WeaponCatalog, TerrainCatalog`	Неизменяемые описания переиспользуются.
Bridge	`BattleMap + ConsoleMapRenderer`	Данные карты не зависят от способа вывода.
Composite	`IBattleComponent`	BattleState, Clan и Combatant обходятся единообразно.
Command	`IPlayerCommand`	Каждое действие находится в отдельном объекте.
Strategy	`AI controllers, stats modifiers`	Поведение выбирается во время выполнения.
State	`CombatantState`	Idle, Moving, Attacking, Defending, Stunned, Dead.
Observer	`BattleEventPublisher + BattleLog`	Боевая логика не зависит от отображения журнала.
Chain of Responsibility	`DamageStage chain`	Каждая стадия меняет только свою часть урона.
Template Method	`AiTurnTemplate`	Общий алгоритм AI с расовыми hook-методами.
Memento	`BattleMementoFactory`	F5/F9 сохраняет состояние без сериализации сервисов.

9. Готовая последовательность показа преподавателю

Шаг	Файл / класс	Что происходит
1	`Program.cs`	Показать минимальную точку входа.
2	`GameCompositionRoot.cs`	Показать отдельное связывание зависимостей.
3	`BattleRuntimeFactory.cs`	Кратко объяснить, как создаются сервисы одного боя.
4	`BattleBuilder.cs`	Показать создание карты 50x50 и трёх кланов.
5	`IClanFactory.cs` + `ClanFactoryBase.cs`	Показать Abstract Factory и Factory Method.
6	`CombatantPrototype.cs`	Показать Prototype и бонусы главы.
7	`TurnScheduler.cs`	Объяснить инициативу и начало хода.
8	`ReachableAreaFinder.cs`	Объяснить Дейкстру и жёлтые контуры доступных клеток.
9	`WindowsConsoleInputReader.cs`	Показать Adapter клавиатуры и мыши.
10	`MoveCommand.cs` -> `MovementService.cs`	Нажать на клетку и показать движение по шагам.
11	`AttackCommand.cs` -> `CombatService.cs`	Атаковать и показать передачу цели.
12	`DamagePipeline.cs` + четыре Stage	Объяснить Chain of Responsibility.
13	`BattleEventPublisher.cs` -> `BattleLog.cs`	Показать строку урона в нижнем журнале.
14	`AiTurnTemplate.cs` + три AI	Показать Template Method и Strategy.
15	`BattleMementoFactory.cs`	F5, выполнить действие, F9 и показать восстановление.
16	`SmashBattle.Tests`	Запустить dotnet test и показать проверку правил без UI.

Команды проверки

```
dotnet build SmashBattle.sln -c Release

dotnet test SmashBattle.sln
```

Финальная фраза: проект разделён так, чтобы новая раса, стадия урона, AI-стратегия, modifier или event subscriber добавлялись отдельным классом без переписывания остальных подсистем.